

# 应用数学导论大作业上机报告

崔正平 2400010714

## 1 问题描述

### 1.1 二维 Poisson 方程的有限元离散格式

考虑区域  $\Omega = (0, 1) \times (0, 1)$  上的变系数 Poisson 方程及其非齐次 Dirichlet 边界条件

$$\begin{cases} -\nabla \cdot (\kappa(x, y) \nabla u) = f(x, y), & (x, y) \in \Omega, \\ u = g(x, y), & (x, y) \in \partial\Omega, \end{cases} \quad (1)$$

其中扩散系数  $\kappa$  满足  $0 < \kappa_{\min} \leq \kappa \leq \kappa_{\max}$ ,  $\kappa, f, g$  可视为从给定函数分布中采样得到的输入函数, 例如

$$\kappa \sim \mu_{\kappa}, \quad f \sim \mu_f, \quad g \sim \mu_g.$$

对每一组样本  $(\kappa, f, g)$ , 问题 (1) 的弱形式为: 求  $u \in H^1(\Omega)$ , 且  $u|_{\partial\Omega} = g$ , 使得

$$\int_{\Omega} \kappa \nabla u \cdot \nabla v \, dx \, dy = \int_{\Omega} f v \, dx \, dy, \quad \forall v \in H_0^1(\Omega).$$

划分均匀三角网格  $\mathcal{T}_h$ , 网格宽度为  $h = \frac{1}{N_h}$ , 设  $\{\phi_i\}_1^{(N_h-1)^2}$  为内部节点对应的基函数, 其在各个三角形上分片线性, 在内部格点  $\mathbf{x}_j$  取值为  $\phi_i(\mathbf{x}_j) = \delta_{ij}$ .

为处理边界项, 记分片线性函数  $u_{g,h}(\mathbf{x})$  为  $g$  的分片线性插值, 满足

$$u_{g,h}(\mathbf{x}_j) = \begin{cases} g(\mathbf{x}_j), & \mathbf{x}_j \in \partial\Omega, \\ 0, & \mathbf{x}_j \in \Omega. \end{cases}$$

令  $u_h = u_{g,h} + \sum_{i=1}^{(N_h-1)^2} U_i \phi_i$ , 于是我们只需要求解  $\mathbf{U} = (U_1, \dots, U_{(N_h-1)^2})^T$  为如下线性方程组的解

$$\mathbf{A}(\kappa) \mathbf{U} = \mathbf{b}(\kappa, f, g),$$

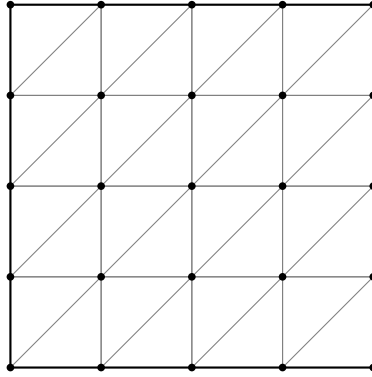
其中

$$\begin{aligned} a_{ij} &= \int_{\Omega} \hat{\kappa}(x, y) \nabla \phi_j(x, y) \cdot \nabla \phi_i(x, y) \, dx \, dy, \\ b_i &= \int_{\Omega} \hat{f}(x, y) \phi_i(x, y) \, dx \, dy - \int_{\Omega} \hat{\kappa}(x, y) \nabla u_{g,h} \cdot \nabla \phi_i \, dx \, dy. \end{aligned}$$

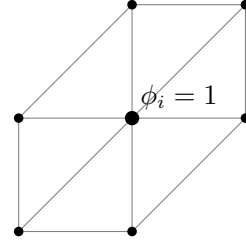
在上式的数值积分中, 为了方便计算, 我们采用分片线性函数  $\hat{\kappa}, \hat{f}$  对  $\kappa, f$  做近似, 在格点处 (包括内部和边界格点) 取值满足  $\hat{\kappa}(\mathbf{x}_j) = \kappa(\mathbf{x}_j), \hat{f}(\mathbf{x}_j) = f(\mathbf{x}_j)$ .

若采用齐次 Dirichlet 边界条件  $g = 0$ , 则  $u_{g,h} = 0$ , 右端项简化为

$$b_i = \int_{\Omega} \hat{f}(x, y) \phi_i(x, y) dx dy.$$



均匀三角网格  $\mathcal{T}_h$



节点基函数支撑区域示意

图 1: 二维均匀三角形网格与有限元节点基函数示意图.

为求解线性方程组  $\mathbf{A}\mathbf{U} = \mathbf{b}$ , 传统 Gauss-Seidel 迭代可写成

$$\mathbf{U}^{(m+1)} = \mathbf{U}^{(m)} + (\mathbf{D} - \mathbf{L})^{-1}(\mathbf{b} - \mathbf{A}\mathbf{U}^{(m)}), \quad \mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U},$$

其中  $\mathbf{D}, \mathbf{L}, \mathbf{U}$  分别为  $\mathbf{A}$  的对角、负的严格下三角与负的严格上三角部分. 本大作业希望采用算子学习的方法加速传统 Gauss-Seidel 迭代求解.

## 1.2 MIONet 神经算子

### 1.2.1 核心思路

MIONet (Multiple-Input Operator Network) 是一类用于学习多输入算子的神经网络. 对于我们考虑的 Poisson 问题 (1), 目标算子可记为

$$\mathcal{G} : (\kappa, f, g) \mapsto u,$$

在实际训练中, 对训练数据集  $\mathcal{D}$  中的训练数据  $\kappa, f, g$ , 我们无法将其作为函数直接输入, 只能输入其在若干采样点  $\{\mathbf{s}_\ell^\kappa\}_{\ell=1}^{m_\kappa}, \{\mathbf{s}_\ell^f\}_{\ell=1}^{m_f}, \{\mathbf{s}_\ell^g\}_{\ell=1}^{m_g} \subseteq \overline{\Omega}$  上的取值, 这些采样点称为 sensors, 对输入函数作离散观测

$$\boldsymbol{\kappa}_s = (\kappa(\mathbf{s}_1^\kappa), \dots, \kappa(\mathbf{s}_{m_\kappa}^\kappa)), \quad \mathbf{f}_s = (f(\mathbf{s}_1^f), \dots, f(\mathbf{s}_{m_f}^f)), \quad \mathbf{g}_s = (g(\mathbf{s}_1^g), \dots, g(\mathbf{s}_{m_g}^g)).$$

我们希望训练一个网络  $\mathcal{M}_\theta : \mathbb{R}^{m_\kappa} \times \mathbb{R}^{m_f} \times \mathbb{R}^{m_g} \times \Omega \rightarrow \mathbb{R}$  对解算子近似:

$$\mathcal{M}_\theta(\boldsymbol{\kappa}_s, \mathbf{f}_s, \mathbf{g}_s)(\mathbf{x}) \approx \mathcal{G}(\kappa, f, g)(\mathbf{x}) = u(\mathbf{x}), \quad \forall \mathbf{x} \in \bar{\Omega}.$$

同样限于有限数据, 我们无法在所有  $\Omega$  中的点上进行优化, 同样只能取  $\Omega$  的一个子集  $\{\mathbf{q}_\ell\}_{\ell=1}^{m_s} \subseteq \bar{\Omega}$ . 因此我们实际是优化如下误差:

$$\begin{aligned} & \arg \min_{\theta} \frac{1}{M} \sum_{(\kappa, f, g, u) \in \mathcal{D}} \|\mathcal{M}_\theta(\boldsymbol{\kappa}_s, \mathbf{f}_s, \mathbf{g}_s) - u\|_{L^2(\Omega)}^2 \\ & \approx \arg \min_{\theta} \frac{1}{M m_s^2} \sum_{(\kappa, f, g, u) \in \mathcal{D}} \|\mathcal{M}_\theta(\boldsymbol{\kappa}_s, \mathbf{f}_s, \mathbf{g}_s)(\mathbf{q}) - u(\mathbf{q})\|_2^2 \\ & = \arg \min_{\theta} \frac{1}{M m_s^2} \sum_{(\kappa, f, g, u) \in \mathcal{D}} \sum_{\mathbf{q}_\ell} |\mathcal{M}_\theta(\boldsymbol{\kappa}_s, \mathbf{f}_s, \mathbf{g}_s)(\mathbf{q}_\ell) - u(\mathbf{q}_\ell)|^2, \end{aligned} \quad (2)$$

其中  $M = |\mathcal{D}|$  为数据集大小.

### 1.2.2 网络架构

MIONet 使用多个 BranchNet 编码不同输入函数, 并使用 TrunkNet 编码空间点. 为了保证学到的解算子关于右端项线性, BranchNet <sub>$f_g$</sub>  被设计为单层全连接网络且不含偏置项. 而 BranchNet <sub>$\kappa$</sub>  和 TrunkNet 选取有一定自由度, 最简单的取法就是若干层全连接网络和激活函数 (如 ReLU) 的复合. 网络架构有如下形式, 其中  $p$  是最后一层隐藏层的宽度:

$$\mathcal{M}_\theta(\boldsymbol{\kappa}_s, \mathbf{f}_s, \mathbf{g}_s)(\mathbf{x}) = \sum_{i=1}^p (B_\kappa(\boldsymbol{\kappa}_s))_i (B_{fg}(\mathbf{f}_s, \mathbf{g}_s))_i (T(\mathbf{x}))_i.$$

### 1.2.3 训练数据

训练数据可按如下方式构造:

1. 从函数分布  $\mu_\kappa, \mu_f, \mu_g$  中采样  $M$  组问题数据  $(\kappa^{(n)}, f^{(n)}, g^{(n)})$ ;
2. 取网格宽度为  $H$ , 对每组数据组装其对应的 Poisson 有限元问题的刚度矩阵  $\mathbf{A}^{(n)}$  与右端项  $\mathbf{b}^{(n)}$ , 用直接法或充分收敛的迭代法得到参考解  $\mathbf{U}^{*(n)}$ , 由其组装成有限元解  $u_h^{(n)} = u_{g,h}^{(n)} + \sum_{i=1}^{(N_H-1)^2} U_i^{*(n)} \phi_i$  用来作为  $(\kappa^{(n)}, f^{(n)}, g^{(n)})$  对应的问题的真解  $u^{(n)}$  的近似;
3. 采样恰当的 sensors  $\{\mathbf{s}_\ell^\kappa\}_{\ell=1}^{m_\kappa}, \{\mathbf{s}_\ell^f\}_{\ell=1}^{m_f}, \{\mathbf{s}_\ell^g\}_{\ell=1}^{m_g}, \{\mathbf{q}_\ell\}_{\ell=1}^{m_s}$  用于生成训练数据. 为便于计算, 我们只考虑将其简单取成网格  $\mathcal{T}_H$  的格点, 此时  $m_\kappa = m_f = m_s = (N_H + 1)^2$  为全部格点,  $m_g = 4N_H$  为边界格点. 由此生成数据集  $\mathcal{D} = \{(\boldsymbol{\kappa}_s^{(n)}, \mathbf{f}_s^{(n)}, \mathbf{g}_s^{(n)}, u_h^{(n)}(\mathbf{q}))\}$ .

得到数据集后, 直接采用 SGD 优化残量 (2) 即可.

## 1.3 混合迭代

混合迭代的算法流程如下:

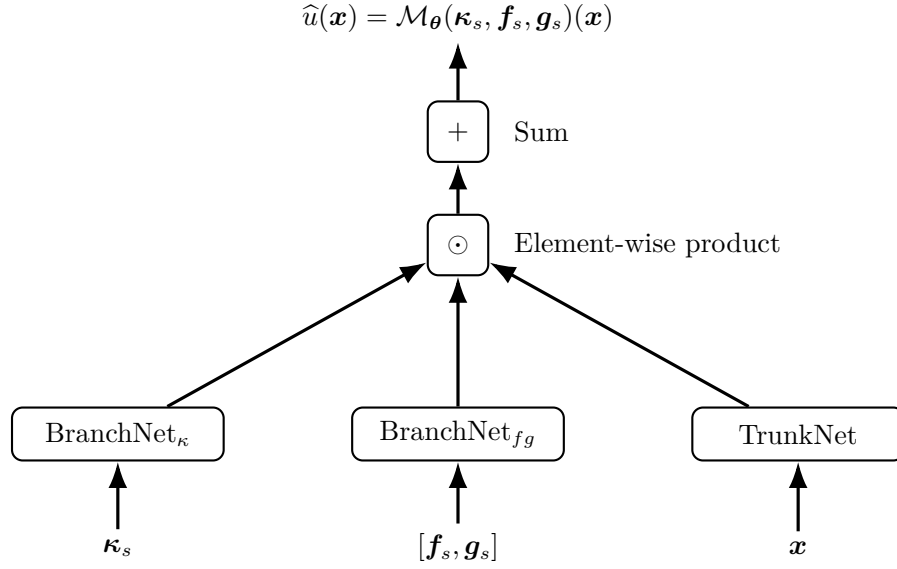


图 2: MIONet 架构示意图.

1. 对给定的问题  $(\kappa, f, g)$ , 生成网格宽度为  $h$  的有限元矩阵  $\mathbf{A}$  与右端项  $\mathbf{b}$ , 并选取恰当的初值作为迭代起点. 初值可以随意选取, 例如选 0 作为初值, 也可选取神经网络  $\mathcal{M}_{\theta}(\kappa_s, f_s, g_s)$  在网格格点上的取值作为初值, 即  $U_i^{(0)} = \mathcal{M}_{\theta}(\kappa_s, f_s, g_s)(x_i)$ .
2. 开始 Gauss-Seidel 迭代求解方程组. 每间隔  $K$  步, 采用神经网络做一次校正. 设当前残量为  $\mathbf{r}^{(n)} = \mathbf{b} - \mathbf{A}U^{(n)}$ , 我们希望得到误差  $\mathbf{e}^{(n)} = \mathbf{A}^{-1}\mathbf{r}^{(n)} = U_h - U^{(n)}$ . 我们需要从  $\mathbf{r}^{(n)}$  构造出用于输入网络的  $f^{(n)}$  和  $g^{(n)}$ , 可以选取

$$f_i^{(n)} = \begin{cases} 0, & x_i \in \partial\Omega \cap \mathcal{T}_H, \\ \frac{1}{h^2} r_i^{(n)}, & x_i \in \Omega \cap \mathcal{T}_H, \end{cases}, \quad g^{(n)} = \mathbf{0},$$

于是混合迭代算法就可以写成

$$\begin{cases} U^{(n+1)} = U^{(n)} + (D - L)^{-1}(\mathbf{b} - \mathbf{A}U^{(n)}), & n \not\equiv 0 \pmod{K}, \\ U_i^{(n+1)} = U_i^{(n)} + \mathcal{M}_{\theta}(\kappa_s, f^{(n)}, \mathbf{0})(x_i), & n \equiv 0 \pmod{K}, \quad x_i \in \mathcal{T}_h. \end{cases}$$

## 1.4 大作业要求

### 1.4.1 基本目标

1. 实现二维 Poisson 方程的有限元离散和 Gauss-Seidel 迭代求解算法, 通过绘制误差曲线等方法检验算法正确性.
2. 实现 MIONet 神经算子, 取恰当的  $H$  生成数据用于网络训练, 通过绘制数值解和真解的图像等方法检验算法正确性.  $N_H$  可选取为 16, 32, 64 等适当大小.
3. 实现基于 MIONet 的加速方法, 取恰当的网格宽度  $h$ , 在未参与训练的测试样本中选取测试函数  $(\kappa, f, g)$ ,

分别采用 Gauss-Seidel 迭代和混合迭代求解问题, 比较传统迭代与神经算子加速迭代在收敛速度、相对误差和运行时间上的差异. 网格宽度可取满足  $N_h = N_H, 2N_H, 4N_H$  等等, 停机标准可取为  $\|\mathbf{AU}^{(n)} - \mathbf{b}\|_2 / \|\mathbf{b}\|_2 \leq 10^{-6}$ .

### 1.4.2 探索内容

有诸多可以自行探索的内容, 以下简单罗列一些仅供参考, 可以做一些相关的文献调研, 也可提出自己的想法或设计一些实验.

1. 不同参数的选择对结果的影响, 例如采样点  $\mathbf{s}, \mathbf{q}$ , 训练数据集规模  $M$ , 网络深度, 训练数据网格宽度  $H$  ( $H$  可以不和  $h$  成比例, 此时混合迭代第 2 步中的  $\mathbf{f}$  需要通过线性插值得到), SGD 中的 batch size, 优化器的选择, 学习率的大小等等.
2. 混合迭代对不同迭代法的加速效果. 例如不同松弛因子为  $\omega$  下的松弛 Jacobi 迭代法.
3. 网络架构对算法的影响. 例如设计 BranchNet 和 TrunkNet 时, 如果不采用全连接网络, 而采用卷积网络或是其他架构 (例如 ResNet, Transformer); 在选取激活函数时, 不选取 ReLU 而是其他函数; 或是取消  $f, g$  对应网络的线性与无偏的限制; 甚至直接替换神经算子, 采用 FNO 或其他神经算子.
4. 训练数据集对算法的影响. 例如在训练时只选取  $\kappa \equiv 1$ , 即只在标准 Poisson 问题上进行网络训练, 在测试时能否泛化到一般的  $\kappa$  对应的问题上. 又如果从不同的函数分布中采样训练数据  $(\kappa, f, g)$  (例如采样为多项式等). 或者考虑直接采样  $(\kappa, u)$  直接计算右端项  $(f, g)$ .
5. 混合迭代加快 Gauss-Seidel 迭代收敛的原理分析.

## 2 硬件、系统和环境配置

### 2.1 硬件配置

- 神经网络训练硬件配置 (AutoDL 租赁)
  - GPU: RTX 3090 (24GB)
  - CPU: 14 vCPU Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz
  - 内存: 90GB
- 其他实验硬件配置 (本地)
  - CPU: Intel(R) Core(TM) Ultra 5 125H
  - 内存: 32GB

### 2.2 系统和环境配置

- 系统: Linux (本地 Windows 11 系统通过 WSL2 运行虚拟机)
- 环境:
  - Python 3.12 (ubuntu 22.04)
  - PyTorch 2.8.0
  - CUDA 12.8
- 依赖库 (均可采用最新版本):

```
- torch
- scipy
- numpy
- matplotlib
- yaml
- os
- sklearn
- sys
- random
- time
- datetime
- itertools
- warnings
```

### 3 超参数和实验设置

#### 3.1 神经网络超参数设置

- $\kappa$ -分支网络: 全连接网络 (FNN), 4 层中间层, 仅输出层无偏置, 有激活函数
- $f, g$ -分支网络: 单层线性层, 无偏置, 无激活函数
- 主干网络: 全连接网络 (FNN), 4 层中间层, 有偏置, 有激活函数
- 特征维度  $p$ : 128
- 中间层宽度: 256
- 激活函数:  $\text{ReLU}(x) = \max(0, x)$
- 损失函数: 均方误差 (Mean Squared Error, MSE), 并在测试时采用相对  $L^2$  误差.

#### 3.2 训练超参数设置

- 批次大小 (Batch Size): 64
- 训练总轮数 (Epochs): 300
- 优化器: AdamW
- 学习率调度策略: CosineAnnealingLR (余弦退火)
- 初始学习率: 0.001
- 权重衰减率 (Weight Decay): 0.0001
- 随机数种子: 0

#### 3.3 数据集生成设置

- 系数生成, 基于高斯过程 (Gaussian Process):

- 扩散系数  $\kappa$  与源项  $f$  : 采用基于 RBF 径向基核的二维高斯随机场生成, 并对  $\kappa$  进行了阈值截断处理 ( $\kappa \geq 0.1$ )
- 边界条件  $g$  : 采用基于 ExpSineSquared 周期核的一维高斯过程生成
- 参考解生成: 每个样本的数值精确解  $u$  均调用基于稀疏 LU 分解的高精度稀疏线性方程组求解器 `scipy.sparse.linalg.spsolve` 得到
- 数据集规模与网格分辨率:
  - 训练集 (Train): 6400 个样本, 网格尺寸  $N_{\text{train}} = 32$
  - 验证集 (Val): 1600 个样本, 用于检验 MIONet 神经算子训练效果, 网格尺寸  $N_{\text{val}} = 32$
  - 测试集 (Test): 1 个样本, 用于混合迭代测试以及其他实验, 网格尺寸  $N_{\text{test}} \in \{32, 64\}$
- 随机数种子: 0

### 3.4 混合迭代测试设置

- 对于混合 (Hybrid) 迭代法, 每  $K = 40$  次传统迭代法后进行 1 次 MIONet 神经算子校正
- 停机条件: 残差的  $L^2$  范数小于  $10^{-12}$
- 传统迭代采用无矩阵 (Matrix-free) 更新格式
- 测试耗时基于原生 Python 环境测定, 绝对耗时受底层 Python 的 GIL 锁及解释器开销影响显著。

### 3.5 传统迭代法设置

- 每种传统迭代法对于每个方格的更新逻辑, 都是根据当前与它相邻的 4 个方格内的旧值及其系数计算得到新值, 并与旧值按照松弛因子进行加权, 但方格更新的顺序互不相同:
  - GS 迭代: 按照行优先的字典序逐一更新, 存在强数据依赖, 只能严格串行
  - RB-GS 迭代: 将所有方格按照奇偶性红黑染色, 然后先同时更新所有红格, 再同时更新所有黑格, 可以并行
  - Jacobi 迭代: 同时更新所有方格, 可以并行
- 松弛因子:
  - Damped GS 迭代与 Damped RB-GS 的松弛因子取为 1.8
  - Damped Jacobi 迭代的松弛因子取为 0.8
- 对于 RB-GS 迭代, 还可以引入 Filter (滤波) 操作, 在每个大循环内通过较少次数的 Damped Jacobi 迭代磨光红格与黑格之间的高频误差, 具体设置为:
  - Hybrid RB-GS (Filtered): 一次大循环内 36 次 RB-GS + 4 次 Damped Jacobi
  - Hybrid Damped RB-GS (Filtered): 一次大循环内 28 次 Damped RB-GS + 12 次 Damped Jacobi

## 4 有限元离散格式与 GS 迭代验证

我们构造了一个具有解析解的二维变系数 Poisson 方程

$$\begin{cases} -\nabla \cdot (\kappa(x, y) \nabla u) = f(x, y), & (x, y) \in \Omega, \\ u = g(x, y), & (x, y) \in \partial\Omega, \end{cases}$$

其中

$$\kappa(x, y) = 1 + 0.5xy,$$

$$g(x, y) = \sin(\pi x) \cos(\pi y),$$

$$f(x, y) = -\left[\frac{\pi}{2}y \cos(\pi x) \cos(\pi y) - \frac{\pi}{2}x \sin(\pi x) \sin(\pi y) - 2\pi^2 \left(1 + \frac{1}{2}xy\right) \sin(\pi x) \cos(\pi y)\right],$$

对应的解析真解为  $u(x, y) = \sin(\pi x) \cos(\pi y)$ . 由此我们验证了线性有限元离散格式的收敛阶为 2, 以及 GS 迭代算法求解线性方程组的正确性.

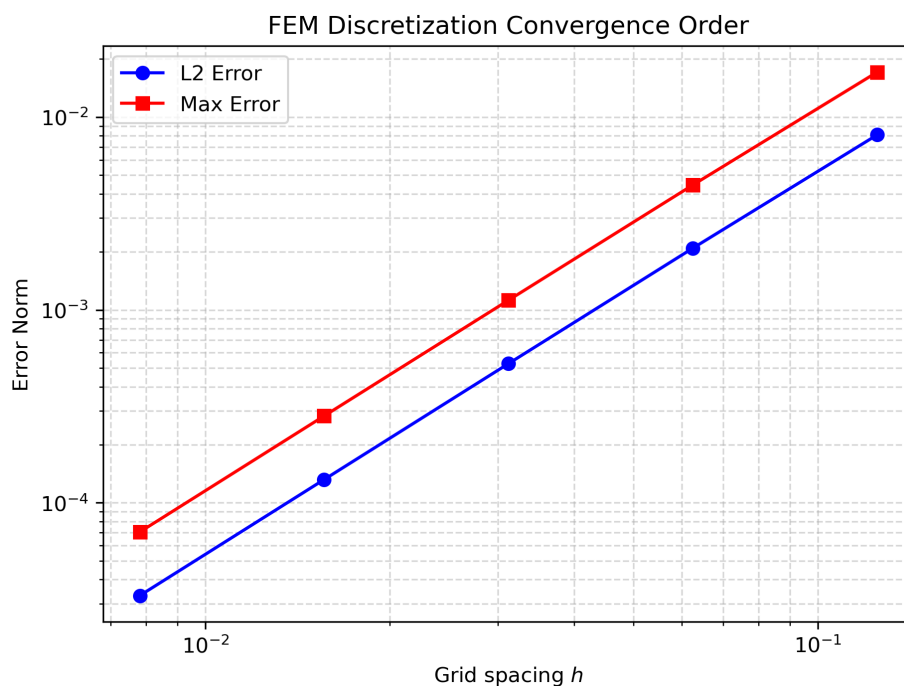


图 3: 有限元离散格式在不同网格尺寸下的误差



| $N$ | $h$    | $L^2$ 误差                | 收敛阶  | $L^\infty$ 误差           | 收敛阶  |
|-----|--------|-------------------------|------|-------------------------|------|
| 9   | 0.1250 | $8.0940 \times 10^{-3}$ | -    | $1.7124 \times 10^{-2}$ | -    |
| 17  | 0.0625 | $2.0916 \times 10^{-3}$ | 1.95 | $4.4472 \times 10^{-3}$ | 1.95 |
| 33  | 0.0312 | $5.2703 \times 10^{-4}$ | 1.99 | $1.1235 \times 10^{-3}$ | 1.98 |
| 65  | 0.0156 | $1.3201 \times 10^{-4}$ | 2.00 | $2.8214 \times 10^{-4}$ | 1.99 |
| 129 | 0.0078 | $3.3019 \times 10^{-5}$ | 2.00 | $7.0568 \times 10^{-5}$ | 2.00 |

表 1: 有限元离散格式在不同网格尺寸下的误差与收敛阶

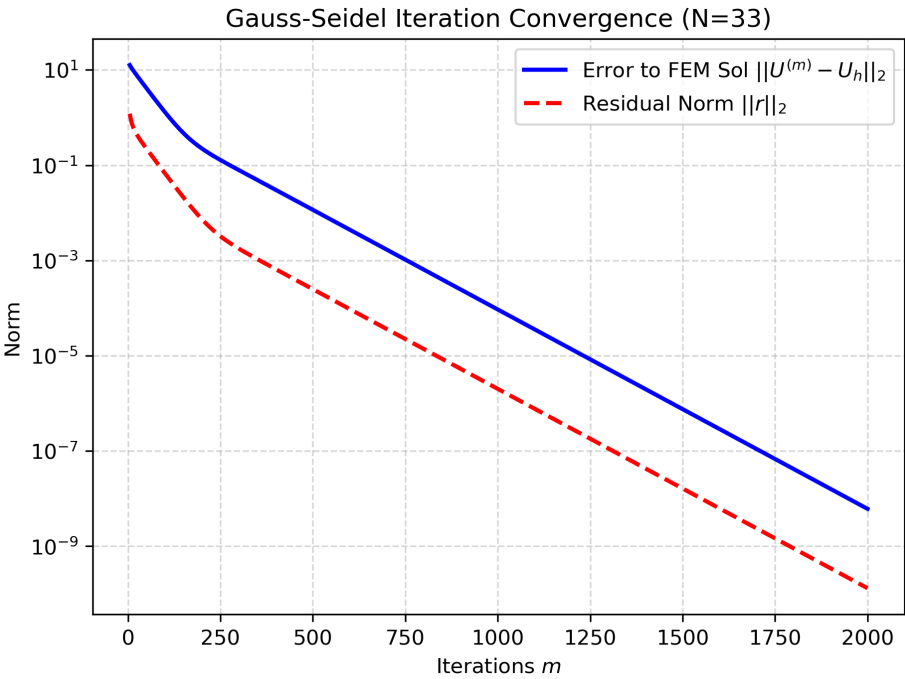


图 4: Gauss-Seidel 迭代在  $N = 33$  下的收敛性

## 5 MIONet 神经算子超参数对比实验

我们针对中间层层数、特征维度、中间层宽度、激活函数、批次大小、学习率、权重衰减率等超参数进行了较为全面的对比实验, 最终选定的设置为中间层层数为 4、特征维度为 128、中间层宽度为 256、激活函数为 ReLU、批次大小为 64、学习率为 0.001、权重衰减率为 0.0001, 此时取得了 2.2294% 的较优测试相对  $L^2$  误差.

5.1 实验结果

| 中间层层数 | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|-------|----------------|----------------|----------------|-------------|
| 2     | 2.8264%        | 3.0168%        | 0.6328M        | 2.91        |
| 3     | 2.5049%        | 2.7495%        | 0.7644M        | 3.45        |
| 4     | <b>2.0349%</b> | <b>2.2294%</b> | <b>0.8960M</b> | <b>4.26</b> |
| 5     | 2.3737%        | 2.6653%        | 1.0276M        | 5.24        |

表 2: 在特征维度为 128、中间层宽度为 256、激活函数为 ReLU、批次大小为 64、学习率为 0.001、权重衰减率为 0.0001 的设置下, 神经算子在不同的中间层层数下的训练结果

| 特征维度       | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|------------|----------------|----------------|----------------|-------------|
| 64         | 3.7370%        | 4.0804%        | 0.7853M        | 4.13        |
| 128        | 2.0349%        | 2.2294%        | 0.8960M        | 4.26        |
| <b>256</b> | <b>1.9435%</b> | <b>2.1798%</b> | <b>1.1174M</b> | <b>4.13</b> |

表 3: 在中间层层数为 4、中间层宽度为 256、激活函数为 ReLU、批次大小为 64、学习率为 0.001、权重衰减率为 0.0001 的设置下, 神经算子在不同的特征维度下的训练结果

| 中间层宽度      | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|------------|----------------|----------------|----------------|-------------|
| 128        | 2.6620%        | 2.8490%        | 0.4276M        | 4.18        |
| <b>256</b> | <b>2.0349%</b> | <b>2.2294%</b> | <b>0.8960M</b> | <b>4.26</b> |
| 512        | 2.1369%        | 2.3973%        | 2.4225M        | 4.31        |

表 4: 在中间层层数为 4、特征维度为 128、激活函数为 ReLU、批次大小为 64、学习率为 0.001、权重衰减率为 0.0001 的设置下, 神经算子在不同的中间层宽度下的训练结果

| 激活函数        | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|-------------|----------------|----------------|----------------|-------------|
| <b>ReLU</b> | <b>2.0349%</b> | <b>2.2294%</b> | <b>0.8960M</b> | <b>4.26</b> |
| Tanh        | 11.2248%       | 10.7272%       | 0.8960M        | 4.28        |
| Sigmoid     | 33.9479%       | 33.4323%       | 0.8960M        | 4.17        |

表 5: 在中间层层数为 4、特征维度为 128、中间层宽度为 256、批次大小为 64、学习率为 0.001、权重衰减率为 0.0001 的设置下, 神经算子在不同的激活函数下的训练结果

| 批次大小      | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|-----------|----------------|----------------|----------------|-------------|
| 32        | 3.2224%        | 3.6077%        | 0.8960M        | 8.20        |
| <b>64</b> | <b>2.0349%</b> | <b>2.2294%</b> | <b>0.8960M</b> | <b>4.26</b> |
| 128       | 3.2778%        | 3.4528%        | 0.8960M        | 2.22        |

表 6: 在中间层层数为 4、特征维度为 128、中间层宽度为 256、激活函数为 ReLU、学习率为 0.001、权重衰减率为 0.0001 的设置下, 神经算子在不同的批次大小下的训练结果

| 学习率          | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|--------------|----------------|----------------|----------------|-------------|
| 0.01         | 94.7442%       | 88.8321%       | 0.8960M        | 4.40        |
| <b>0.001</b> | <b>2.0349%</b> | <b>2.2294%</b> | <b>0.8960M</b> | <b>4.26</b> |
| 0.0001       | 4.0407%        | 4.2340%        | 0.8960M        | 4.22        |

表 7: 在中间层层数为 4、特征维度为 128、中间层宽度为 256、激活函数为 ReLU、批次大小为 64、权重衰减率为 0.0001 的设置下, 神经算子在不同的学习率下的训练结果

| 权重衰减率         | 训练相对 $L^2$ 误差  | 测试相对 $L^2$ 误差  | 参数量            | 训练时长 (min)  |
|---------------|----------------|----------------|----------------|-------------|
| 0.001         | 2.0270%        | 2.2317%        | 0.8960M        | 4.24        |
| <b>0.0001</b> | <b>2.0349%</b> | <b>2.2294%</b> | <b>0.8960M</b> | <b>4.26</b> |
| 0.00001       | 2.0841%        | 2.3014%        | 0.8960M        | 4.32        |

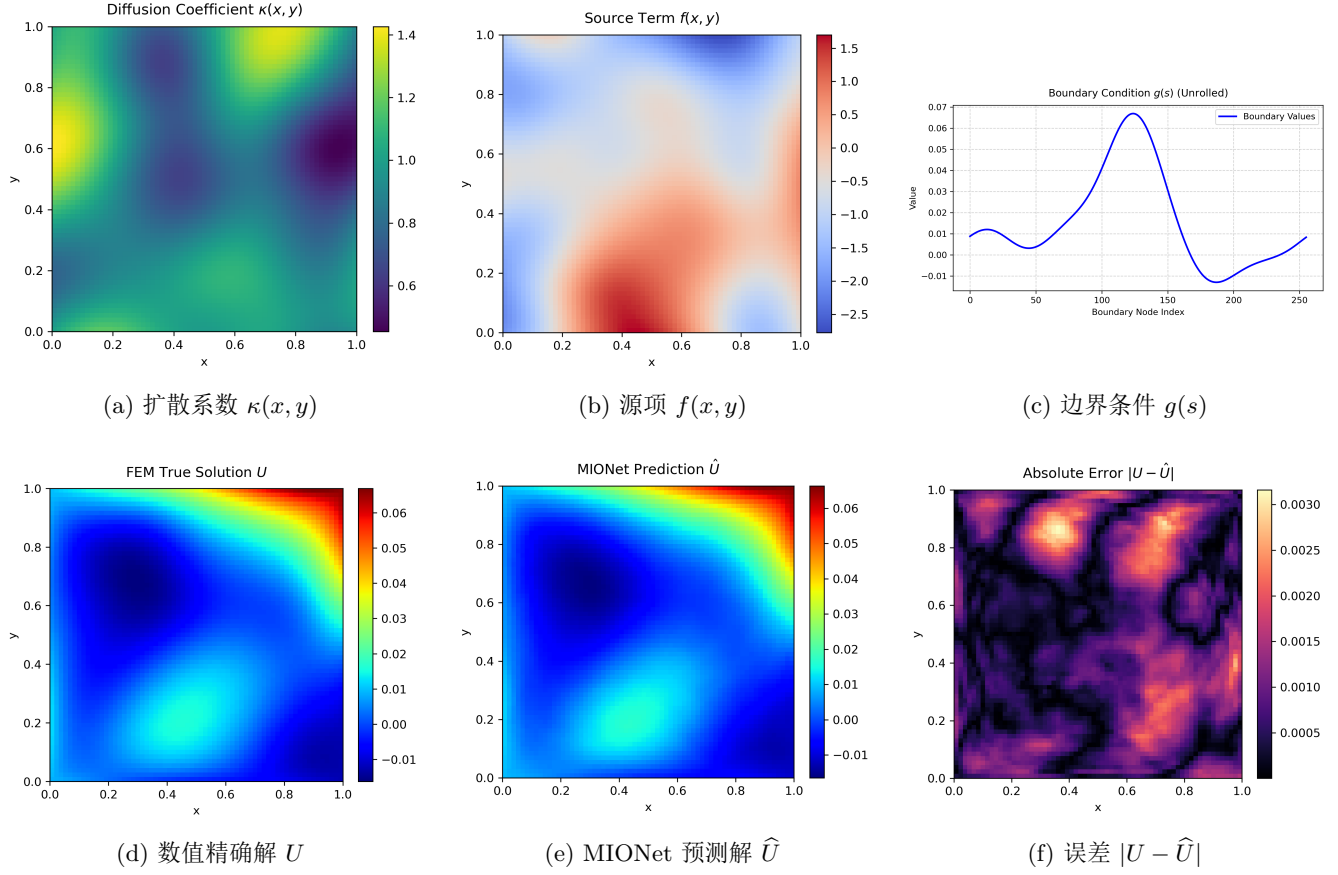
表 8: 在中间层层数为 4、特征维度为 128、中间层宽度为 256、激活函数为 ReLU、批次大小为 64、学习率为 0.001 的设置下, 神经算子在不同的权重衰减率下的训练结果

## 5.2 结果分析

- **中间层层数对比:** 中间层层数 (网络深度) 直接决定了神经算子的非线性表征能力. 随着层数从 2 层增加到 4 层, 神经算子的拟合能力稳步提升, 测试相对  $L^2$  误差从 3.01% 降至 2.22%. 然而, 当层数进一步加深至 5 层时, 误差反而出现回升. 这表明过深的 FNN 开始出现优化困难 (如梯度传播衰减) 或轻微的过拟合现象, 因此 4 层是平衡表征能力与优化难度的最佳深度
- **特征维度对比:** 特征维度代表了神经算子最终输出时空间基函数的数量, 增加特征维度意味着模型能够使用更多的基函数去逼近目标算子. 实验表明, 随着特征维度从 64 提升至 256, 测试误差显著下降, 这是因为更大的特征维度更有利于捕捉解流形中的高频分量
- **中间层宽度对比:** 增加中间层宽度可以提升单层的特征提取能力. 当宽度由 128 提升至 256 时, 测试误差显著下降; 但继续拓宽至 512 时, 神经算子的参数量激增至 2.42M (发生严重的过参数化), 导致泛化能力下降, 测试误差反弹至 2.39%. 因此 256 的中间层宽度既保证了足够的特征提取能力, 又避免了冗余参数带来的过拟合
- **激活函数对比:** 在求解二维变系数 Poisson 方程时, ReLU 展现出了绝对的优势. Tanh 和 Sigmoid 激活函数在网络加深至 4 层后极易发生梯度消失的现象, 且其饱和特性不适合拟合具有较大取值范围的物理场, 导致误差极高 (Sigmoid 甚至完全无法收敛, 误差高达 33%), 凸显了非饱和激活函数在偏微分方程算子学习中的重要性
- **批次大小对比:** 批次大小不仅影响训练时间, 更显著影响优化效果. 当批次大小为 128 时, 虽然训练极快, 但由于单步梯度的方差过小, 模型容易陷入尖锐的局部极小值, 泛化误差偏高; 当批次大小为 32 时, 梯度噪声过大, 导致收敛震荡. 64 的批次大小在训练效率与梯度稳定性之间取得了最佳的平衡
- **学习率对比:** 学习率决定了梯度下降的步长. 当学习率过大 (0.01) 时, 优化过程发生剧烈震荡甚至发散, 导致误差高达 88%; 当学习率过小 (0.0001) 时, 模型在 300 个 Epoch 内处于欠拟合状态, 未能收敛至最优解. 0.001 的学习率是配合 CosineAnnealingLR 调度器的最佳初始学习率
- **权重衰减率对比:** 权重衰减作为  $L^2$  正则化手段, 对于神经算子有一定影响. 由于  $f, g$ -分支网络被取成无偏置的单层线性层, 极易过拟合训练集数据. 实验表明, 设置适中的 0.0001 的权重衰减率能提供较优的正则化惩罚, 既有效抑制了过拟合, 又保留了神经算子足够的拟合容量

## 6 MIONet 神经算子预测验证

为了直观展示 MIONet 神经算子的网格无关性和泛化能力, 我们选取了一个  $N = 64$  的样本, 将其输入 (扩散系数  $\kappa$ 、源项  $f$ 、边界条件  $g$ ) 以及神经算子预测结果与有限元数值精确解的对比如图 5 所示.

图 5: MIONet 神经算子在  $N = 64$  的网格上的输入与预测结果对比

## 7 混合迭代测试

采用训练好的 MIONet 神经算子 ( $N_{\text{train}} = 32$ ) 直接对  $N_{\text{test}} = 32, 64$  的样本进行测试.

7.1 实验结果

| 迭代法                                   | $N_{\text{test}} = 32$ |               | $N_{\text{test}} = 64$ |               |
|---------------------------------------|------------------------|---------------|------------------------|---------------|
|                                       | 大循环数                   | 总耗时 (s)       | 大循环数                   | 总耗时 (s)       |
| GS                                    | 64                     | 2.7537        | 194                    | 33.8464       |
| Hybrid GS                             | 7                      | 0.6062        | 12                     | 2.4754        |
| Damped GS                             | 6                      | 0.2537        | 22                     | 3.9158        |
| Hybrid Damped GS                      | 7                      | 0.3287        | 10                     | 1.9521        |
| RB-GS                                 | 65                     | 0.1444        | 196                    | 0.5832        |
| Hybrid RB-GS                          | 11                     | 0.0674        | 173                    | 2.4346        |
| Hybrid RB-GS (Filtered)               | 7                      | 0.0354        | 13                     | 0.1731        |
| Damped RB-GS                          | 6                      | 0.0131        | 23                     | 0.0702        |
| <b>Hybrid Damped RB-GS (Filtered)</b> | <b>4</b>               | <b>0.0197</b> | <b>6</b>               | <b>0.0875</b> |
| Jacobi                                | 128                    | 0.0740        | 386                    | 0.4937        |
| Hybrid Jacobi                         | 102                    | 0.3097        | 357                    | 4.4638        |
| Damped Jacobi                         | 160                    | 0.1030        | 482                    | 0.5833        |
| Hybrid Damped Jacobi                  | 9                      | 0.0317        | 26                     | 0.3162        |

表 9: 不同传统迭代与 MIONet 混合迭代在不同网格下的性能对比

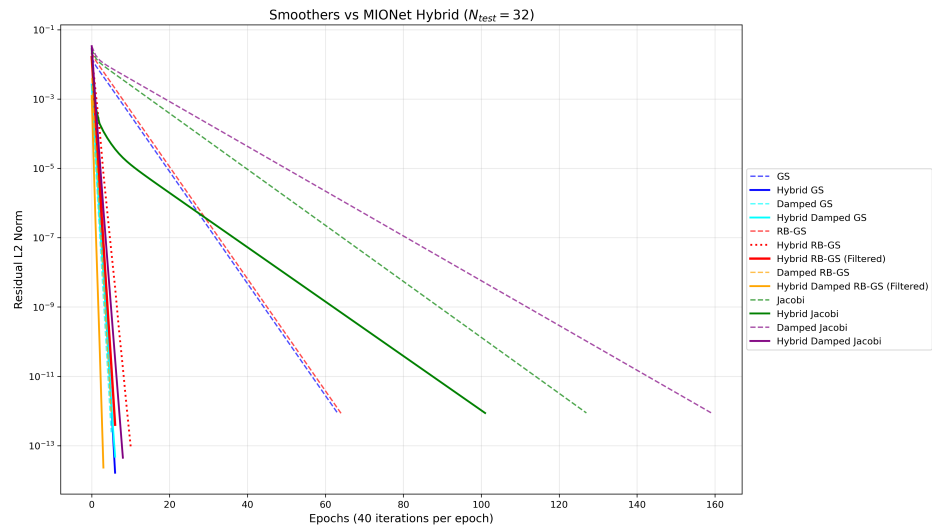


图 6: 不同传统迭代与 MIONet 混合迭代在  $N_{\text{test}} = 32$  下的残差曲线

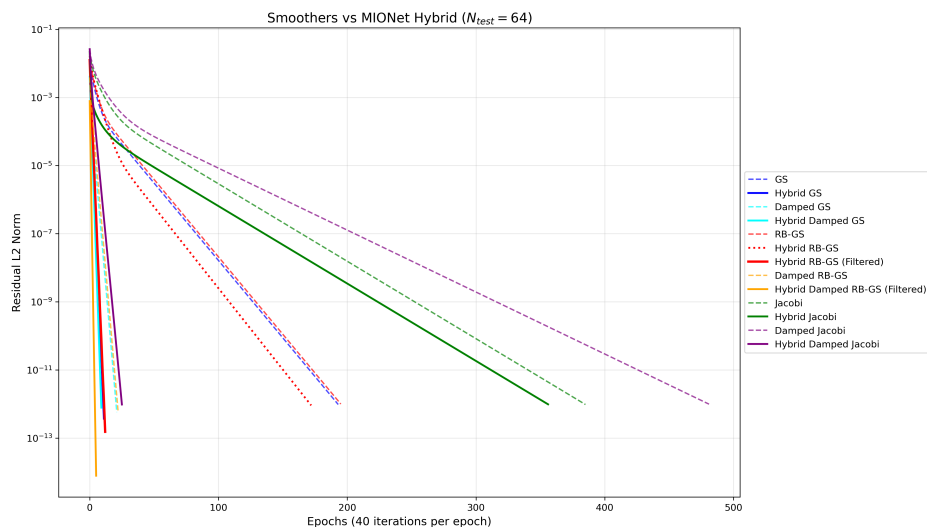


图 7: 不同传统迭代与 MIONet 混合迭代在  $N_{\text{test}} = 64$  下的残差曲线

为了完整对比上述所有传统迭代与 MIONet 混合迭代在消除高频与低频误差时的频域表现, 我们依次绘制了每种迭代法在 200 个迭代步 (5 个大循环) 内的频谱误差热力图。

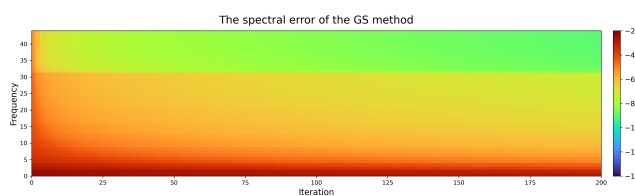


图 8: GS 迭代方法的频谱演化图

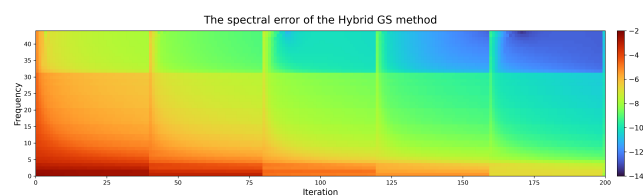


图 9: Hybrid GS 混合迭代的频谱演化图

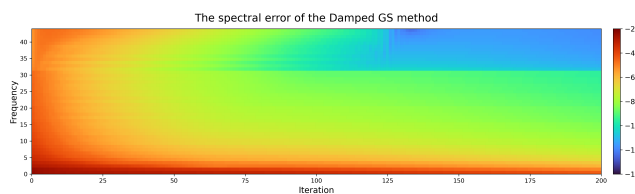


图 10: Damped GS 迭代方法的频谱演化图

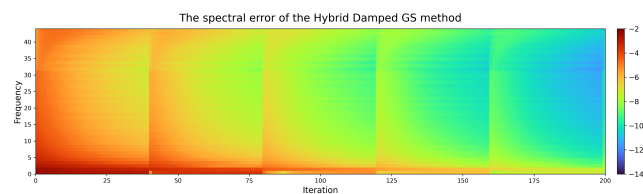


图 11: Hybrid Damped GS 混合迭代的频谱演化图

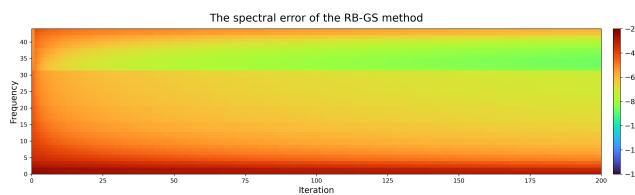


图 12: RB-GS 迭代方法的频谱演化图

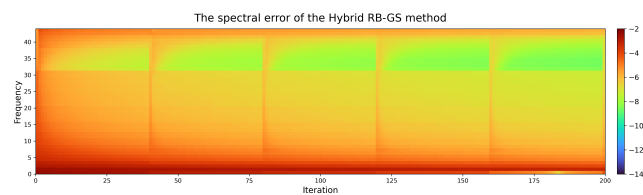


图 13: Hybrid RB-GS 混合迭代的频谱演化图

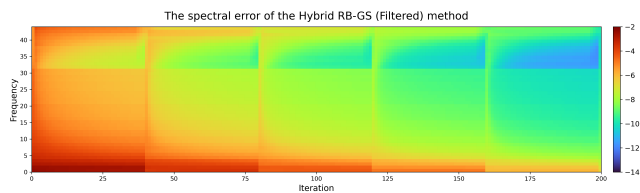


图 14: Hybrid RB-GS (Filtered) 混合迭代的频谱演化图

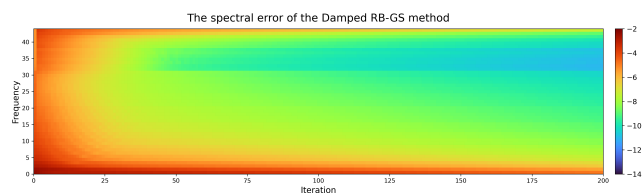


图 15: Damped RB-GS 迭代方法的频谱演化图

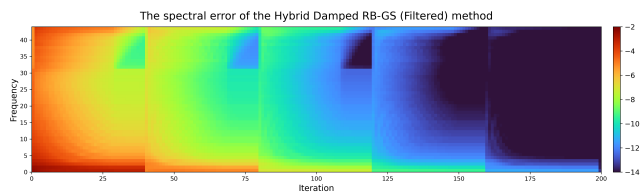


图 16: Hybrid Damped RB-GS (Filtered) 的频谱演化图

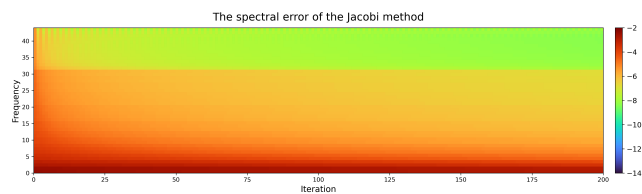


图 17: Jacobi 迭代方法的频谱演化图

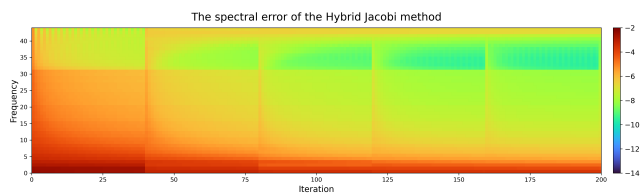


图 18: Hybrid Jacobi 混合迭代的频谱演化图

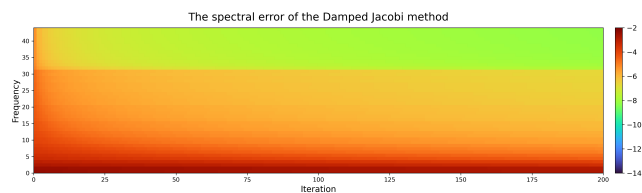


图 19: Damped Jacobi 迭代方法的频谱演化图

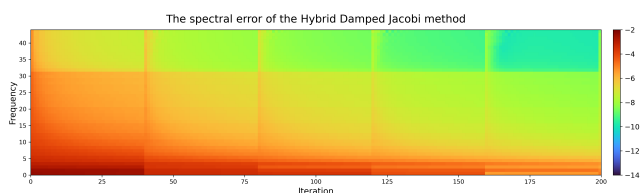


图 20: Hybrid Damped Jacobi 混合迭代的频谱演化图

在混合迭代算法中, 修正周期  $K$  是一个很关键的超参数. 为探究其对收敛速度的影响, 我们在  $N_{\text{test}} = 64$  网格下, 记录了在不同的  $K$  下 Hybrid GS 方法所需的总迭代步数与总耗时, 见表 10.



| $K$       | 总迭代步数      | 总耗时 (s)       |
|-----------|------------|---------------|
| 5         | 发散         | 发散            |
| 10        | 480        | 2.6904        |
| 20        | 460        | 2.2771        |
| <b>30</b> | <b>450</b> | <b>2.1849</b> |
| 40        | 480        | 2.2392        |
| 60        | 540        | 2.5874        |
| 80        | 560        | 2.8661        |
| 100       | 700        | 3.1057        |
| 120       | 720        | 3.5224        |

表 10: Hybrid GS 方法在不同的修正周期  $K$  下的总迭代步数与总耗时对比

## 7.2 结果分析

### 7.2.1 不同迭代法与 MIONet 加速效果对比

- 在原生 Python 环境下, RB-GS 与 Jacobi 迭代由于高度可并行性, 其单步耗时远低于只能严格串行的 GS 迭代
- 引入适当的松弛因子一方面能显著加快传统迭代 (GS, RB-GS) 的收敛速度, 另一方面也能显著提高传统迭代的磨光性, 对于 Jacobi 迭代, 阻尼版本的 MIONet 加速效果显著优于无阻尼版本
- 在  $N_{\text{test}} = 64$  下, RB-GS 迭代的 MIONet 加速效果甚微 (大循环数仅从 196 降至 173), 这是由于红黑染色更新会在空间中引入额外的高频误差, 神经算子对这种高频误差极不敏感, 导致校正效果很差. 在引入 Damped Jacobi 作为滤波器后, 混合迭代的大循环数骤降至 13 次, 加速效果显著
- 结合松弛因子、滤波器与 MIONet 神经算子, Hybrid Damped RB-GS (Filtered) 展现出了绝对的优势, 在  $N_{\text{test}} = 64$  时仅需 6 个大循环即可收敛, 相比 GS 迭代在迭代速度上加速了 32.3 倍, 在耗时上加速了 386 倍

### 7.2.2 不同迭代法的频谱误差分析

- 观察传统迭代 (GS, Jacobi) 的频谱误差热力图可以发现, 高频误差能够较快地衰减, 但低频误差的衰减极其缓慢, 收敛速度很慢
- MIONet 作为一种学习偏微分方程解流形的全局神经算子, 具有“频谱偏置”特性, 天然擅长拟合平滑的低频成分. 在频谱误差热力图中, 每隔  $K = 40$  步迭代 MIONet 校正时, 低频误差被分摊到各个频率上, 有效加速了低频误差的衰减. 这印证了混合迭代法的优势: 传统迭代负责磨光高频误差, 而 MIONet 神经算子负责消除低频误差, 两者实现了完美的互补
- 观察 RB-GS 迭代及其引入 MIONet 加速和滤波后的频谱误差热力图可以发现, RB-GS 迭代擅长消除中频误差, 而高频误差衰减得很慢 (这源自红黑染色更新), 因此 MIONet 加速后效果甚微. 但再引入滤波

操作后, 高频误差也能有效消除, 最终实现了最快的收敛速度

### 7.2.3 神经算子修正周期的影响

- 当修正周期过小时, 神经算子校正过于频繁时, 传统迭代尚未能有效消除高频误差, 使得高频残差被过早送入 MIONet, 最终导致误差急剧累积, 即迭代矩阵的谱半径大于 1, 混合迭代发散
- 当修正周期过大时, 随着  $K$  的不断增大, 混合迭代逐渐退化为传统迭代, 神经算子的低频校正效果甚微, 收敛速度很慢
- 当修正周期取值在  $30 \sim 40$  附近时, 传统迭代的磨光性与 MIONet 的低频校正优势达到了平衡, 此时总迭代步数最低, 且调用神经算子的耗时也被合理摊薄, 实现了最快的求解速度

## 8 总结与展望

本次大作业系统性地研究了基于算子学习 (MIONet) 加速求解二维变系数 Poisson 方程的有限元离散格式的传统迭代法. 通过有限元离散与传统迭代法 (GS、RB-GS、Jacobi 等) 的实现, 并结合高斯随机场数据集的生成与神经网络超参数对比实验, 我们成功实现了具有较强泛化能力与网格无关性的神经算子.

在混合迭代的核心实验中, 我们从收敛速度、耗时与频谱误差三个角度, 深入分析了 MIONet 与传统迭代的互补机制. 数值结果不仅验证了该混合迭代的优势 (最高实现 386 倍耗时加速), 还研究了红黑染色更新逻辑、松弛因子以及修正周期  $K$  对混合迭代的影响.

未来, 该方案可进一步拓展至三维复杂几何区域或非线性偏微分方程的求解中; 在网络架构方面, 也可探索引入 FNO 或者 Transformer 等其他神经网络架构.

## 致谢

在本次大作业的完成过程中, 特别感谢 Gemini 3.1 Pro 在代码调试、文本润色等方面提供的帮助.